Guía para la defensa Proyecto 3

Grupo 15 · Pochoco SoC · FPGA a ASIC

Basada en el código y las corridas reales de este repositorio. La defensa presencial vale 1,5 puntos y debe realizarla el grupo.

Recorrido de cinco minutos

Explicar la procedencia del Proyecto 2; recorrer asic_top → pochoco_soc_asic → Espino/periféricos/memoria; mostrar las dos memorias y el mecanismo de arranque; presentar área/timing/signoff; terminar con la comparación RTL/netlist y las limitaciones.

Resultados que conviene tener a mano

Síntesis jerárquica	A	B
Área de celdas µm²	147.741,350	613.575,962
Instrucciones	107	107
Datos usados	0	0

Las mediciones finales por corrida están en results/summary.csv; abrir corners.csv para responder preguntas de setup/hold. Evitar memorizar un número de una corrida antigua y atribuirlo al layout final.

Register file, ALU y contador

Register file: asic/rtl/espino_register_file.v; x0 constante, 15 registros escribibles, dos lecturas síncronas, lectura antes de escritura. ALU: rtl/espino_core/espino_alu.v; la ruta crítica debe verificarse en el reporte, no suponerse. Contador: asic/rtl/cycle_counter.v y captura led_cycle_q en asic/rtl/pochoco_periph.v. La extracción jerárquica separa sus áreas; las capturas de OpenDB/KLayout ubican sus celdas.

Preguntas probables y respuestas

1. ¿De qué versión parte el proyecto?

De main de Proyecto2AC, revisión 67d0c48; se conserva el historial en un clon independiente. dev-game era anterior. GitHub de Proyecto 2 no fue modificado.

2. ¿Qué hace asic_top?

Expone i_Clk, rst_ni, cuatro botones, cuatro LED, catorce señales de segmentos y SPI. Instancia pochoco_soc_asic y no añade pads.

3. ¿Por qué se eliminó el POR FPGA?

por_cnt dependía de una inicialización al configurar la FPGA. Un ASIC necesita un reset eléctrico explícito y condiciones de desactivación verificadas.

4. ¿Qué significa RV32E aquí?

Datos de 32 bits y 16 registros arquitectónicos. x0 siempre devuelve cero. La implementación del grupo no ejecuta desplazamientos ni instrucciones M.

5. ¿Qué hace el assembler?

En dos pasadas resuelve etiquetas y codifica las instrucciones del grupo. La extensión --rom-dir emite casos Verilog y constantes de reset desde el mismo código máquina.

6. ¿Cuánto programa hay?

107 instrucciones de 32 bits: 428 bytes. generate_game.py y test_rom.py lo verifican en la imagen real.

7. ¿Cuánta RAM de datos usa el juego?

Cero palabras: los lw/sw usan x1=0x80000000 y acceden a MMIO. El estado vive en registros. El testbench también contó cero accesos a RAM.

8. ¿Por qué existe una palabra de datos?

Es la reserva mínima de la implementación, en 0x1AC. Permite probar RAM y byte enables. Debe distinguirse capacidad implementada de uso real.

9. ¿Cuál es la diferencia entre ROM y RAM?

A fija instrucciones en lógica combinacional y permite escribir solo su RAM de datos. B permite cambiar instrucciones y datos, y restaura el programa al aplicar reset.

10. ¿Cómo arranca B?

Sus 108 palabras reciben valores explícitos en la rama de reset sintetizable. No hay readmemh ni bootloader SPI. El costo de esos FF y reset se incluye en el área.

11. ¿Qué es un flip-flop?

Un elemento que almacena un bit. El banco tiene 15 × 32 bits de estado arquitectónico; B añade miles de bits de programa y multiplexores, por eso aumenta el área.

12. ¿Por qué no usar lectura combinacional en el register file?

Espino espera una lectura síncrona mediante ops_valid_q. Cambiar la latencia sin cambiar el protocolo podría ejecutar operandos incorrectos.

13. ¿Cómo se garantiza x0?

La escritura a dirección cero se ignora y ambos puertos retornan cero cuando su dirección registrada corresponde a x0.

14. ¿Qué hace la ALU?

Suma, resta, XOR, OR, AND y comparaciones con/sin signo. En espino_alu.v los desplazamientos están deshabilitados. El assembler los rechaza para evitar resultados erróneos.

15. ¿Qué hace cycle_counter?

Incrementa 32 bits por ciclo y vuelve a cero con reset. El periférico copia su valor en led_cycle_q en el mismo flanco que escribe LED.

16. ¿El tiempo medido coincide exactamente con pulsar?

El inicio sí queda capturado en el flanco del LED. El fin se lee por software después del sondeo del botón. El estímulo a 127 ciclos resultó en 136 medidos: existe latencia de detección y lectura.

17. ¿Qué sucede si el contador da la vuelta?

La resta modular de 32 bits conserva el intervalo si transcurren menos de 2^32 ciclos. A 10 MHz, ese intervalo es aproximadamente 429,497 segundos.

18. ¿Cómo se adaptaron los tiempos?

Para 10 MHz: espera 30.000.000, décima 1.000.000 y divisor promedio 10.000.000. La imagen abreviada usa 300, 100 y 1000 solo para simulación.

19. ¿Qué inconsistencia había en main?

El game.s de main carga 250.000.000 ciclos, mientras que su game.hex carga 75.000.000 y coincide con el informe. Solo cambian dos palabras. Se preservó esa procedencia y se regeneraron fuente/hex de Proyecto3AC para 10 MHz.

20. ¿Cómo calcula el promedio?

Acumula ciclos sin truncar en x3 y divide la suma por diez décimas de reloj mediante restas. A 10 MHz el divisor es 10.000.000.

21. ¿Qué se probó con el botón incorrecto?

Después de un acierto se envía un botón distinto del objetivo. El display muestra EE y la ronda se reinicia, manteniendo x2 y x3.

22. ¿Qué es síntesis?

Transforma RTL en una red de celdas GF180. No produce por sí sola posiciones ni cables físicos ni demuestra DRC/LVS.

23. ¿Qué son placement, CTS y routing?

Placement ubica celdas; CTS distribuye clock; routing conecta sus terminales con metales/vías. Sus buffers y parasíticos modifican área y timing.

24. ¿Qué significa utilización del 40 %?

La relación objetivo entre área lógica y área del core al crear el floorplan. No equivale a la ocupación final tras buffers, taps y filler.

25. ¿Qué son setup y hold?

Setup exige que el dato llegue antes del flanco receptor; hold exige que permanezca estable después. Reducir el período no soluciona hold.

26. ¿Qué son WNS y TNS?

WNS es el peor slack negativo y vale cero cuando no hay fallas. TNS suma slacks negativos. Para conocer margen positivo se consulta ws, además de WNS/TNS.

27. ¿Por qué verificar nueve corners?

Se combinan tres condiciones de RC con TT, SS y FF. Un resultado típico no demuestra que setup y hold cierren en todos ellos.

28. ¿Cómo se identifica la ruta crítica?

En el final, max_ss_125C_4v50/max.rpt muestra u_pochoco.u_ram._604_/Q a u_pochoco.u_ram._648_/D: instr_rdata[4] a ram_b_rdata[16]. El retardo combinacional Q a D es 54,650438 ns. Cruza decodificación/ejecución y lectura de memoria; no es el contador ni necesariamente el bloque de mayor área.

29. ¿Qué optimización se probó?

Se registró dirección/selección del puerto de datos de A para separar el cálculo de dirección de la decodificación ROM. La salida conserva su comportamiento al flanco; se compararon 2.000 ciclos aleatorios y el juego.

30. ¿Cómo se calcula la frecuencia máxima?

En el barrido con síntesis/PnR, 60 ns cierra y 50 ns falla. Sobre el layout final fijo, 59 ns cierra los nueve corners y 58 ns falla: 1000/59 = 16,949153 MHz observados, con resolución de 1 ns. Se distinguen ambos métodos. La entrega usa 100 ns, 10 MHz.

31. ¿Qué significa DRC limpio?

Las geometrías cumplen las reglas revisadas por el verificador ejecutado. Aquí se reporta Magic; la configuración del PDK no proporcionó un deck de KLayout y no se afirma que ese chequeo haya pasado.

32. ¿Qué verifica LVS?

Netgen compara la conectividad extraída del layout con la red esperada. Cero discrepancias no reemplaza las pruebas funcionales del juego.

33. ¿Qué es antena?

Durante fabricación puede acumularse carga en metal conectado a compuertas. El flujo repara y vuelve a verificar; los reportes finales deben mostrar cero nets/pins violadores o una justificación explícita.

34. ¿Las capturas demuestran dónde está la ALU?

Sí cuando las marcas usan las cajas de instancias u_alu extraídas de OpenDB. Los colores por sí solos no son prueba; cell_locations.json permite reproducir la selección.

35. ¿Cómo se calcula el costo?

Se divide el área del die en µm² por un millón y se multiplica por US$23.000/mm². No se incluyen máscaras, yield, prueba ni encapsulado.

36. ¿Cuál fue la decisión de mayor impacto en área?

La síntesis jerárquica comparable da A 147.741,3504 y B 613.575,9616 µm²: B ocupa aproximadamente 4,15 veces el área. El programa reescribible en FF y sus multiplexores explican el aumento; la reserva de datos del juego es mínima.

37. ¿Qué demuestra la simulación de netlist?

Ejecuta celdas estándar con la misma configuración y compara estímulos y resultados por ciclo. No se anotó SDF; STA post-PnR es la evidencia de retardos.

38. ¿Qué no debemos afirmar en la defensa?

No afirmar fabricación garantizada, DRC KLayout aprobado, equivalencia formal exhaustiva o una frecuencia no medida. Declarar los límites de slew, fanout y capacitancia de A y B, las etapas pendientes y la resolución de la búsqueda de frecuencia.

39. ¿Cómo publicamos el proyecto?

El alumno verifica su identidad y ejecuta personalmente commit y push. Codex no los ejecutó. El remoto proyecto2 tiene push deshabilitado en este clon.

40. ¿B se comparó bajo las mismas condiciones físicas que A?

Sí: A_p100_u30_v2_resume y B_p100_u30_v2_resume_20261102T153658Z usan 100 ns y 30 % de utilización inicial, la misma biblioteca y programa. El die medido es 547.483,000 µm² para A y 2.158.790,000 µm² para B. Esta comparación no cambia el final obligatorio de A a 100 ns y 40 %. Las pruebas B al 40 % que conservaron un short se documentan como incompletas.

41. ¿Cuál fue la frecuencia medida al comparar las dos memorias?

Sobre sus layouts fijos, con STA en nueve corners y resolución de 1 ns: A pasa a 57 ns (17,544 MHz), y B a 91 ns (10,989 MHz). El período inmediatamente inferior falla en cada caso. Son límites observados del layout fijo, distintos del barrido de síntesis/PnR.

Archivos para demostrar, no solo explicar

scripts/verify.py: pruebas ejecutables.
results/synthesis/A_sim_hier/gate_comparison.json: comparación real de netlist.
results/optimization/memory_equivalence.log: prueba comparativa de memoria.
results/runs/*/corners.csv: corners de timing.
docs/figures/: vistas reales del layout.
docs/CHECKLIST_ENTREGA.md: estado contra la rúbrica.
docs/Informe_Proyecto3.pdf: análisis completo.